

Assignment 9 : Adding Custom Instructions to Microwatt – Part B

1. List the VHDL files that need to be created or modified to add a new custom instruction to Microwatt. Which one is entirely new?

- **decode_types.vhdl** : The updated to add the new instructions and opcode group.
- **predecode.vhdl** : Adds support for the new instructions as well.
- **decode1.vhdl** : which decodes the primary opcodes.
- **decode2.vhdl** : Fortunately, one of the values for the three-bit output mux that selects between the execution units was not yet used.
- **custom_unit.vhdl** : In this file the execution unit the modification of the execution of the custom instruction is made.
- **execute1.vhdl** : to add the custom execution unit

The custom_unit.vhdl this file is newly created file.

2. Explain the role of custom_unit.vhdl. What are its inputs and outputs?

custom_unit.vhdl : In this file the execution unit the modification of the execution of the custom instruction is made.

The input ports are ra, rb, rc and insn.

- The ra, rb and rc are 64-bit wide
- The insn is 32-bit wide. Since the instructions is 32-bit encoded format

The output ports are result.

- The out is the result or destination values sent out.

3. In custom_unit.vhdl, how does the design select between the four different custom operations? Which bits of the instruction encode this choice?

In the file custom_unit.vhdl the design select between the four different custom operations is by insn.

insn(1 downto 0)	Operation	Description
"00"	custom_addbusat	Byte-wise saturating add
"01"	custom_subbusat	Byte-wise saturating subtract

6. What is the role of `execute1.vhdl`, and why would it need to be modified to invoke the new `custom_unit`?

`execute1` performs integer operations and branches. It can send memory operations to the LSU or floating-point work to the FPU. Bypass paths return recent results to decode without waiting for register-file writeback.

The reason is it doing the ALU memory operations hence the `custom_unit` is making certain changes in the execution wise hence we need to modify accordingly the things also.

7. Explain the patch-based workflow used to apply the custom instruction changes to the original VHDL files (e.g., `decode1.vhdl.patch`). Why is this approach useful?

In general, the patch-based workflow gives the release notes or what got updated or what got changed in the source codes. A kind of history or something.

8. Why can't the existing GCC/toolchain assembler recognize the new custom instructions directly? How can they still be used in C programs?

The reason is in the existing GCC or toolchain assembler doesn't support the newly introduced instructions as they are not supported at all. Hence we build around the custom C function

9. Why does the icache predecode instructions into a 36-bit form, and how does this interact with adding new opcodes?

The 36-bit cache format is not a compressed instruction format. It is a **10-bit predecoded instruction ID** (`insn_code`) plus the **original instruction's low 26 bits**. That width fits FPGA block RAMs well. The predecoder classifies instructions when cache data is filled, so `decode1` can use the cached ID to select its control-table entry instead of repeating the opcode/subopcode lookup's

On a cache hit, the I-cache reconstructs a 32-bit instruction for downstream use:

- It restores the top six opcode bits from the `insn_code` and retains the low 26 bits from the cached instruction.
- `decode1` receives both the reconstructed instruction and its `insn_code`; the ID selects the decode controls, while instruction fields provide operands, registers, and immediates. Illegal instructions retain their original instruction bits for reporting.

10. What future extension is proposed for adding a custom unit at the SoC level — where would it attach, and what module would need to change?

The future extension proposed for adding a custom unit at the SoC level is :

- When we add custom units or blocks will be connected to the wishbone bus.
- The soc.vhdl files and additional newly introduced vhdl files and interconnection and all stuff.